

Chapitre 2 Variables

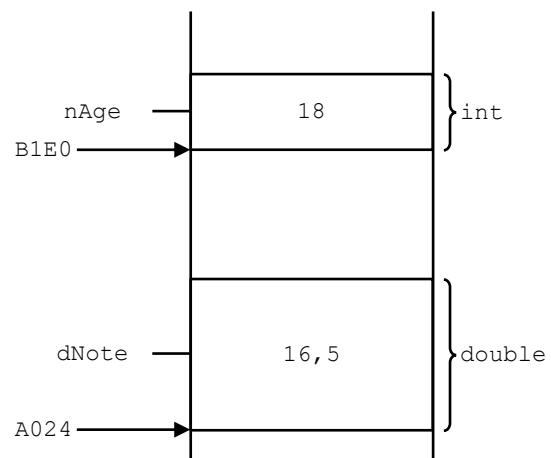
La majorité des programmes informatiques utilisent des données (par exemple le numéro d'un jour de semaine, la note d'un élève, l'âge d'un patient, le numéro d'un compte bancaire, la température d'un matériau, etc.). Dans un programme, les variables servent à réserver un espace dans la mémoire pour stocker des valeurs.

La représentation imagée d'une variable est un espace mémoire dans lequel peut être stockée une valeur :

- Cet emplacement est identifié par une étiquette : c'est le nom, ou **identificateur**, de la variable
- Dans le programme, l'identificateur de la variable est directement utilisé pour accéder à l'espace mémoire de l'ordinateur. Cet espace mémoire est repéré par un numéro appelé **adresse** de la variable (noté en base 16, ou hexadécimal).
- La nature de la valeur qui peut être contenue dans l'emplacement est définie par le **type** de la variable
- A chaque type est associée une **portée**, qui fixe les valeurs minimum et maximum que peuvent prendre les valeurs contenues dans la variable

Le schéma ci-contre illustre l'occupation de deux variables dans la mémoire de l'ordinateur, l'une entière, l'autre réelle :

- `nAge` et `dNote` représentent les identificateurs des deux variables
- 18 et 16,5 sont les valeurs stockées dans les deux variables
- `int` et `double` fixent les types des deux variables
- `B1E0` et `A024` sont les emplacements physiques des deux variables dans la mémoire, encore appelés adresses des deux variables
- Selon leur type, les variables n'occupent pas forcément le même espace en mémoire
-



Si, à ce stade, maîtriser la notion d'adresse mémoire des variables n'est pas absolument nécessaire, retenez que c'est indispensable pour utiliser les tableaux et les pointeurs (voir chapitres correspondants).



1. Documentez-vous sur la numérotation hexadécimale.
2. A partir du schéma ci-dessus, entraînez-vous à dessiner la représentation mémoire des variables de vos programmes. Cela vous aidera à visualiser et donc à maîtriser les notions de déclaration et d'utilisation des variables dans vos programmes.

Déclaration des variables

Pour utiliser une variable dans un programme, il faut au préalable l'avoir déclarée, au moyen de la syntaxe suivante.

Syntaxe

```
type identificateur = valeur;
```

Exemple

```
int main (void)
{
    int nAge = 18;                // âge de 18 ans
    double dNote = 16.5;          // note de 16,5
    double dTemp = -137.8;        // température de -137,8 degrés
}
```

Remarques

- Déclarer une variable permet d'associer un identificateur (facile à utiliser) à un emplacement mémoire dont la taille est déterminée par le type de la variable (les types seront abordés au prochain chapitre). Déclarer une variable entraîne une réservation de place en mémoire, à l'endroit indiqué par l'adresse de la variable et de dimension fixée par le type.
- Les variables peuvent être déclarées n'importe où dans le programme du moment qu'elles le sont avant leur utilisation. Néanmoins, l'usage est de déclarer les variables au plus près de leur utilisation.

Identificateurs des variables

Les identificateurs des variables sont choisis par le programmeur. Ils peuvent être constitués de n'importe quelle combinaison de caractères alphabétiques ou numériques, mais doivent commencer par une lettre. L'utilisation de caractères accentués est déconseillée, car certains compilateurs ne les acceptent pas. Le caractère souligné `_` est considéré comme une lettre.

Il n'y a pas de taille limite maximale pour les identificateurs, mais ceux-ci doivent être choisis courts et représentatifs de la variable qu'ils identifient.

C++ fait la différence entre les majuscules et les minuscules. Par exemple, `dTemp`, et `dtemp` sont des identificateurs différents.

Initialisation des variables

Il faut prendre l'habitude d'initialiser les variables par des valeurs dès leur déclaration, en leur affectant immédiatement des valeurs. En effet, la mémoire de l'ordinateur a souvent été déjà utilisée préalablement à sa réservation. Elle peut donc contenir des valeurs, très souvent incohérentes. On ne peut supposer qu'une variable déclarée contient une valeur par défaut, par exemple 0.

Utiliser une variable déclarée mais non initialisée conduit quasi-systématiquement à une erreur d'exécution du programme. Il appartient donc au programmeur d'initialiser les variables dès leur déclaration.

Exemple

```
int main (void)
{
    int nAge = 18;           // et non int nAge;
    double dNote = 16.5; // et non double dNote;
    double dTemp = -137.8;   // et non double dTemp;
}
```



Dans vos programmes, prenez et conservez l'habitude d'initialiser systématiquement les variables au moment de leur déclaration.

Affectation de valeurs aux variables

Il est souvent nécessaire de modifier la valeur initiale d'une variable pendant l'exécution du programme. Plusieurs moyens sont disponibles pour cela.

Opérateur d'affectation

La valeur contenue dans une variable peut être modifiée simplement par l'opérateur d'affectation = :

Syntaxe

```
identificateur = valeur;
```

Exemple

```
int main (void)
{
    int nAge = 18;           // âge de 18 ans
    double dNote = 16.5;     // note de 16,5
    double dTemp = -137.8;   // température de -137,8 degrés

    nAge = 24;               // l'âge est maintenant 24 ans
    dNote = 12.5;            // la note devient 12,5
    double dTemp = 273.16;   // la température est 273,16 degrés
}
```

Remarques

- Le membre de gauche de l'opérateur = est nécessairement une variable (≡ emplacement mémoire)
- Le membre de droite de l'opérateur représente nécessairement une variable, soit de manière explicite `nAge = 24;` soit de manière implicite `nAge = nAgeTom;` (dans ce cas, `nAgeTom` est une variable entière qui contient l'âge de Tom). La valeur contenue dans la variable `nAgeTom` sert à initialiser la variable `nAge`.

Opérateur d'extraction du flux

L'opérateur d'affectation modifie le contenu d'une variable par une valeur choisie par le programmeur, avant l'exécution du programme.

Souvent, on souhaite aussi donner à l'utilisateur du programme la possibilité de choisir une valeur de son choix pour l'affecter à une variable, au moment de l'exécution du programme.

En C++, on peut utiliser pour cela le flux d'entrée prédéfini `cin` associé à l'opérateur d'extraction du flux `>>`.

14 ► Variables

Syntaxe

```
cin >> = valeur;
```

Exemple

```
#include <iostream>
using namespace std;

int main (void)
{
    // déclaration et initialisation par 0 de la variable nAge
    int nAge = 0;

    // message d'invite pour que l'utilisateur saisisse son âge
    cout << "Saisissez votre âge : ";

    // saisie de l'âge au clavier, et stockage dans la variable nAge
    cin >> nAge;

    // affichage de l'âge de l'utilisateur
    cout << "L'âge de l'utilisateur est " << nAge << "ans." << endl;
}
```

Remarques

- Le clavier de l'ordinateur est le périphérique associé par défaut au flux d'entrée prédéfini `cin`.
- Les instructions `cin >> nAge;` se lisent : « flux d'entrée `cin` » (`cin`), « opérateur d'extraction du flux d'entrée » (`>>`), « variable `nAge` » (`nAge`), « fin de l'instruction » (`;`).
- `cin` se prononce « c » « in » (et non « quinne » ou « cinne »).



Tapez et exécutez ce programme plusieurs fois jusqu'à disposer de l'expérience pour savoir l'écrire spontanément sans aide ni support écrit.

Opérations sur les variables

Sans attendre un prochain chapitre sur les opérateurs du C++, on peut remarquer que les opérateurs arithmétiques de base réalisent des opérations sur le contenu des variables :

+	:	addition	−	:	soustraction
*	:	multiplication	/	:	division

Exemple : calcul de la surface et de la diagonale d'un champ :

```
#include <iostream>
using namespace std;

int main (void)
{
    double dL = 3.0; // longueur du champ
    double dl = 4.0; // largeur du champ

    // affichage de la surface du champ
    cout << "La surface est : " << dL * dl << endl;

    // affichage de la diagonale du champ
    cout << "La diagonale est : " << sqrt (dL*dL + dl*dl) << endl;
}
```